Recursive self-improvement, gated by evidence

An engine that lets deployed agents improve their own operation — proposing, proving, canarying and shipping their own corrections — measured where the world changes under a long-running agent and outcome learning alone cannot recover

SenseLab Research

https://sense-lab.ai

Preprint v0.1 — development seeds — September 2026

Abstract

Agents that run for months on complex work face a problem no static model solves: the world they operate in changes under them, their tasks never repeat exactly, and every session that acts on what used to be true fails the same way until something in the system changes. Scaling such agents is not a retrieval problem. It is a self-improvement problem: the system must detect that its own knowledge has gone wrong, propose a correction, prove it, release it safely, learn from the result, and do all of this without a person in every loop and without shipping the wrong thing to every agent at once. SenseLab is a recursive self-improvement engine built for that. Outcome evidence changes what agents act on (level 0); a repair agent drafts corrections that must pass a stand-in replay, the customer's own replay and a live canary with a significance test before they ship, reversibly (level 1); the loop's own history feeds its next draft (level 2); and the record it produces trains the next model and the repair prompt itself (level 3). This paper is the first measurement of such a loop. We evaluate the whole engine — never its parts against each other — on a continual benchmark in which the correct action for a task class changes *conditionally* at episode 30 of 100, so that half of the class needs the new rule and half still needs the old one, and no amount of retrying the old action can recover the first attempt. On the development seeds SenseLab takes the new action first in 43 % of moved-half episodes against 5–19 % for a frontier model alone and three vector-store baselines, holding unchanged work at 99 %; a context window carrying every prior note reaches 33 % at 2.3× the cost. Level 1 has not yet added to level 0 on these seeds, and the tables say why: no canary can decide inside a 100-episode cell, and what the proof tiers passed was mostly the unconditional rule. The paper's main contribution is the record of how a preregistered benchmark found ten things about the loop's evidence path — eight defects, seven of them fixed in production between runs and re-measured on the same protocol — and what each says about building self-improving systems that can be trusted: what counts as a regime shift, how a situation is identified, what a fix must be proved against, who grades the proof, and what counts as traffic. Every number here is from development seeds; the confirmatory seeds run once, after the analysis is frozen.

1 Introduction

The hard problem in scaling agents is not capability but longevity. A model that solves a task today is deployed into an operation that runs for months: the policies it applies are rewritten, the dependencies it pins are audited, the workflows it follows are changed by people who never tell it. Its tasks never repeat exactly, so nothing can be memorised; and the failure that matters is not that it cannot find what it knows but that it acts, confidently, on what *used* to be true — and will do so again on every session that reads the same thing, until something in the system changes. A

static model with a static context degrades monotonically in such a world. A person in every loop does not scale.

The path forward is a system that improves its own operation from its own results, and whose improvements compound: recursive self-improvement. It is also the thing everyone is right to be nervous about. A self-modifying system amplifies its own errors as readily as its corrections; a model that drafts fixes from failure evidence will draft wrong ones, and a wrong fix written into what every agent reads damages every session at once. The question is therefore not whether deployed agents should improve themselves — at scale there is no alternative — but what discipline makes it safe enough to run in production: evidence before proposal, proof before release, staged exposure before full exposure, reversibility after, and an audit trail throughout.

SenseLab is an engine built on that discipline, and this paper measures it. Its loop has four levels (§2). At level 0, every session's outcome is attributed to exactly what the agent read and did, and that evidence — validated, contested, discredited — changes what the next session acts on. At level 1, an entry that was validated and now fails is a *regime shift*; a repair agent drafts a correction on a branch with a manifest of the runs it claims to repair and must not break, and the correction must pass a stand-in replay (Tier 1), the customer's own agent replaying the case set (Tier 2) and a live canary on a hashed share of sessions with a significance test (Tier 3) before it ships — reversibly. At level 2, the loop's own history (what it tried on this entry, what passed, what regressed) is evidence for its next draft. At level 3, the record the loop produces trains the next model and the repair prompt itself. The claim is not that the repair agent is clever. The claim is that **a correction may only change behaviour after it has been tested on the runs it claims to repair and has beaten the control arm on live traffic**, and that a system built this way recovers faster than one that can only demote what failed, while shipping fewer wrong corrections than one that ships on a passed test.

We ask, on a benchmark that knows the true rule before and after every change, whether the whole engine recovers faster and further than the frontier model alone, than three vector-store baselines, than a context window carrying the cell's entire history, and than its own ablations; whether it damages what still worked; what it ships; and what it costs. We call the whole system *SenseLab* throughout: the levels are not features to be compared against each other, they are what the engine delivers, and the ablations that switch parts of it off appear in one attribution table and nowhere else.

What this version is. This is a development-seed report (seeds 11 and 23 of six), written while the confirmatory seeds are still to run. It exists because the preregistration (§3) committed to reporting every prediction, met or falsified, and most of the predictions so far were falsified — by defects in the loop's evidence path that the benchmark found and that were fixed in production between runs. Section 5 is the record of that, and we think it is the paper's main contribution: a self-improving system is only as trustworthy as its evidence path, and every one of these defects would have been invisible to a benchmark that measured retrieval. The headline tables (§4) are the state of the engine on its current cut; the confirmatory numbers will differ, and the direction they differ in is itself predicted (§6).

2 The engine under test

Everything measured is production code on the SenseLab SaaS, run against a dedicated Enterprise account; no constant below was tuned for this study, and each is quoted from the deployed source. The harness never decides anything: it seals sessions through the public API, and the loop's own

scheduler proposes, tests, gates and ships. The four levels of the loop, in the order evidence flows through them:

Level 0 — outcome learning: the evidence record. An agent calls `briefing` and `retrieve` before acting, `record_action` and `record_attempt` as it goes, and `commit_outcome` when the task ends. The sealed trace carries the exact read set; the outcome is attributed to it (an entry read by a run that failed after a recorded attempt against it is charged the failure; an entry read by a clean success is credited). Entries carry an *evidence status* — `untested`, `validated`, `contested`, `discredited` — and `retrieve` returns, with the hits, per-action priors (won/lost here on similar tasks) and a recommendation mode: `act`, `explore` (assigned per agent, so peers explore different actions) or `escalate`. That is level 0 alone; it is the row "SenseLab − repair loop" in Table 4.

Regime shift and situation. A *regime shift* is an entry validated at least three times that now fails: a sealed failure whose read set contains such an entry, or a recorded failed attempt against it. A shift is not a first-exposure failure — an entry that was never validated cannot have shifted — and the guard that separates the two was the third defect this study found (§5). Because the same entry can be right in one situation and wrong in another, the agent labels each session with an `attributes.situation`, and the service resolves the label to the agent's canonical situation (a stemmed-token rule, Jaccard ≥ 0.5, with an embedding second stage at cosine ≥ 0.75); the shift, the proof set and the canary are all scoped to the situation. A shift proposes a fix automatically (`auto_propose`, per agent).

Level 1 — the repair loop. Proposal. The repair agent (GPT-5-class, 50k-token budget) reads the failing runs, the entry, and the loop's own history on that entry, and writes a corrective entry (lever `knowledge`) or a conditional procedure with conditions, steps and a verify (lever `procedure`) on memory branch `repair/<fix>`, with a *manifest*: the runs it predicts it repairs, the runs it must not break, and an expected prevention (0.5 by default). A `code` lever is handed off and never auto-shipped.

Tier 1 — stand-in replay. A stand-in model re-runs each manifest case with the branch as memory and tool calls answered from the recorded run; fidelity is the share of steps answered from the record, and a test below the agent's fidelity floor (0.5) does not count as a pass. A test can be `passed`, `failed` or `inconclusive`.

Tier 2 — customer replay. The fix's *case set* — HMAC-signed cases — is delivered to the customer's agent, which re-runs them on the branch and seals the results; the service grades each replay against the case's expectation. For a regime fix the case set is the run the shift fired on, up to three more of the situation's failed runs (the shift; the fix must turn them into successes) and up to three of its passed runs (what still works; the fix must not break them). A case no judge graded is graded on the outcome the replay reports, and — since the last cut — on its attempts: a success reached *after a recorded failed attempt* fails the case (§5, finding 9).

Tier 3 — canary. A passed fix opens a canary: sessions are bucketed by sha256(*fix*:*session*), a share `pct` reads from the branch, writes always go to `main`, and the trace is stamped with its branch. `decide()` requires both arms to reach `min_outcomes` graded runs *in the fix's situation*, guards that the canary's success rate is not below the control's, and runs a one-sided two-proportion z-test on the recurrence of the judged failure ($p < 0.05$). The benchmark maps the production defaults

(10 %, 20 outcomes, 14 days) to `pct`=50, `min_outcomes`=8, timeout 60 sessions, evaluated every 5 graded sessions. One canary per agent at a time.

Ship, verify, revert. Policy `auto_after_canary` merges a passed canary's branch into `main`; `auto_after_replay` (the "− canary" ablation) merges on a passed Tier 2. A feedback contract holds a fix if fewer than 80 % of recent traces carry provenance. Prevention is verified over the following window; a regressed fix is rolled back, restoring the pre-fix versions. Continuous judges (a two-tier `gpt-5-mini`→`gpt-5` rubric pack) grade sealed traffic and open incidents that are the second producer of proposals.

Level 2 — the loop reads its own history. Every fix event on an entry — proposed, tested, replayed, shipped, regressed, rolled back — is written back as evidence on that entry, and the repair agent's next draft on the same entry reads it: what was tried, what passed, what regressed. A loop that cannot see its own record repeats its own mistakes; this one is measured on whether it stops (the two-change protocol of the flagship experiment, not yet run).

Level 3 — the record trains the model and the improver. The traces the loop produces — including the repair trajectories and the cases it pinned — are a training set: Managed Models (`senselab-s1`) are distilled from them and served with the same evidence record as their memory (§4); and the repair prompt itself is A/B tested across proposals, with proven fixes per prompt as the score, which is the improver improving itself. Only the first rung of the first of these is measured in this version.

3 Benchmark and protocol

Preregistration. Hypotheses, arms, metrics, seeds, the dev/confirmatory split and the analysis freeze were registered before the first run (`HYPOTHESIS_RSI.md`, tag `prereg-rsi-v1`); every change since is a dated amendment (a)–(s) appended below the original, each with a prediction and what would falsify it. Seeds 11 and 23 are the development seeds; 37, 41, 59 and 73 run once, after the analysis code is tagged `analysis-freeze-rsi`. Engine changes made between runs are recorded with their deploy cut to the second, and every result carries the engine cut as a covariate.

Environment. Two families of the ten in the benchmark, both with tool-using agents and oracle-graded outcomes: `support` (a helpdesk agent with account, order and export tools; 15 issue classes) and `ci-fix` (an agent repairing failing CI on a repository; 10 failure classes). Every task instance is generated, never repeated, with random identifiers (`opaque_ids`) so nothing can be read off an id. 100 episodes per cell, 300 distractor entries in every store, retry budget 3, one agent identity per cell, GPT-5.5 at low effort, a post-episode reflection note in every arm that has a store.

Change set C. At episode 30 a subset of classes changes *conditionally*: the correct action moves for instances with one attribute value (the *moved half*) and stays for the others (the *kept half*). Half of post-change tasks are drawn from changed classes. Set C was written after the pilots showed that on an unconditional change (set A) outcome learning alone recovers nearly everything a maintainer who knew the rule would (§5, finding 4): the only place a written rule can beat demotion is where demotion has no signal to demote — the entry is still right half the time. Set C is registered by hash; its remaining eight families are written blind before the confirmatory run.

Systems. Frontier alone (no store); context window (every prior note and outcome of the cell in the prompt, most recent first, 24k tokens); pgvector (bge-small, cosine top-5 over notes); pgvector + rerank (the same cross-encoder SenseLab serves); pgvector + outcomes (notes, LLM consolidation every 5 episodes, a per-entry win/loss counter shown to the agent); and SenseLab (§2, policy `auto_after_canary`). Mem0 and Zep are not run (an earlier grid covers them [1]). The ablations — SenseLab − canary (`auto_after_replay`) and SenseLab − repair loop (policy `review`, proposals never approved) — appear in Table 4 only.

Endpoints. First-attempt success on the moved half after the change (primary); first-attempt success on the kept half and on unchanged classes (collateral damage); episode success and escalation; the fix lifecycle (proposed, Tier 1, Tier 2, canary, shipped, reverted); and cost. Infra errors are excluded from rates and reported. Intervals are cell-level bootstrap (2,000 resamples). With four cells per system on development seeds these intervals are wide by construction and are shown so that no reader mistakes a dev-seed difference for a result.

4 Results on the development seeds

4.1 After the rule changes conditionally

system	moved half	kept half	unchanged classes	pre-change	episode success	escalated	$/ep
Frontier alone	17 % (13/75) [0, 41]	38 % (25/65) [11, 78]	66 % (93/140)	61 % (73/120)	84 % (334/400)	16 % (66/400)	0.011
Context window	33 % (25/75) [0, 78]	98 % (64/65) [94, 100]	100 % (140/140)	85 % (102/120)	92 % (366/400)	8 % (34/400)	0.070
pgvector	17 % (13/75) [2, 38]	45 % (29/65) [13, 78]	93 % (130/140)	71 % (85/120)	88 % (353/400)	12 % (47/400)	0.027
pgvector + rerank	5 % (4/75) [1, 9]	86 % (56/65) [78, 95]	94 % (132/140)	79 % (95/120)	90 % (359/400)	10 % (41/400)	0.028
pgvector + outcomes (DIY)	19 % (14/75) [0, 44]	55 % (36/65) [39, 78]	80 % (112/140)	70 % (84/120)	88 % (351/400)	12 % (49/400)	0.035
SenseLab	43 % (32/75) [33, 54]	60 % (39/65) [42, 73]	99 % (138/140)	83 % (100/120)	92 % (370/400)	8 % (30/400)	0.031

Table 1: After the rule changes conditionally (set C, change at episode 30 of 100; GPT-5.5; four cells per system: `support` and `ci-fix` × seeds 11 / 23). First-attempt success on every post-change episode. *Moved half*: changed-class instances whose rule changed; *kept half*: changed-class instances whose rule did not. Brackets: cell-level bootstrap 95% interval. Development seeds; SenseLab on the `post-seam5b` engine cut.

Table 1 is the state of the engine on its `post-seam5b` cut (§5); the run on the current cut is pending (§6). Three things are in it. First, on the moved half every retrieval memory is at or below the frontier model alone: a note that says "do X for class C" is retrieved as confidently after the change as before, and pgvector with reranking — the best retriever — is the *worst* system on the moved half at 5 %, because it retrieves the stale note most reliably. Second, SenseLab is the only system that recovers the moved half while holding the unchanged classes at 99 %; the context window matches it on `ci-fix` (where the whole history fits in 24k tokens and the model can see the change happen) and does nothing on `support` (0/43), at 2.3× the tokens. Third, SenseLab pays on the kept half: 60 % against 86–98 % for the systems that simply keep applying the old rule. That cost is the shape of the mechanism — the entry is marked contested and the agent explores where the old action was still right — and it is the number the repair loop exists to remove: a written conditional rule would recover the moved half *without* giving up the kept one. Whether it does yet is Table 4.

pair	support s11	support s23	ci-fix s11	ci-fix s23	mean	cells won
SenseLab − Frontier alone	+30	+45	+11	+8	+23	4/4
SenseLab − Context window	+30	+45	-26	-31	+5	2/4
SenseLab − pgvector	+26	+45	+11	+15	+24	4/4
SenseLab − pgvector + rerank	+26	+45	+47	+31	+37	4/4
SenseLab − pgvector + outcomes (DIY)	+30	+45	+5	+8	+22	4/4

Table 2: SenseLab minus each system on the moved half, per cell, in percentage points, paired on (family, seed).

system	support: moved	support: kept	support: unchanged	ci-fix: moved	ci-fix: kept	ci-fix: unchanged
Frontier alone	0 % (0/43)	78 % (21/27)	53 % (37/70)	41 % (13/32)	11 % (4/38)	80 % (56/70)
Context window	0 % (0/43)	96 % (26/27)	100 % (70/70)	78 % (25/32)	100 % (38/38)	100 % (70/70)
pgvector	2 % (1/43)	78 % (21/27)	86 % (60/70)	38 % (12/32)	21 % (8/38)	100 % (70/70)
pgvector + rerank	2 % (1/43)	78 % (21/27)	90 % (63/70)	9 % (3/32)	92 % (35/38)	99 % (69/70)
pgvector + outcomes (DIY)	0 % (0/43)	78 % (21/27)	69 % (48/70)	44 % (14/32)	39 % (15/38)	91 % (64/70)
SenseLab	37 % (16/43)	44 % (12/27)	100 % (70/70)	50 % (16/32)	71 % (27/38)	97 % (68/70)

Table 3: By family: first-attempt success on the moved half, the kept half and the unchanged classes.

4.2 Attribution: what level 1 adds to level 0, so far

system	moved half	kept half	unchanged classes	pre-change	episode success	escalated	infra err	$/ep
SenseLab	43 % (32/75) [33, 54]	60 % (39/65) [42, 73]	99 % (138/140)	83 % (100/120)	92 % (370/400)	8 % (30/400)	0	0.031
SenseLab − canary	33 % (25/75) [12, 62]	74 % (48/65) [56, 87]	98 % (135/138)	83 % (98/118)	92 % (366/396)	8 % (30/396)	4	0.031
SenseLab − repair loop	51 % (38/75) [33, 73]	69 % (45/65) [58, 76]	96 % (135/140)	81 % (97/120)	92 % (369/400)	8 % (31/400)	0	0.032

Table 4: Attribution. SenseLab against its own ablations on the same protocol: without the canary (policy `auto_after_replay`, same run) and without the repair loop (policy `review`: proposals never approved; outcome learning alone). This is the only table in which the ablations appear.

On these seeds level 1 does not yet add to level 0. Without the repair loop (policy `review`: the loop proposes, nothing ships, the memory learns from outcomes alone) the moved half is 51 % and the kept half 69 %; with it, 43 % and 60 %; without the canary but with the replay gate, 33 % and 74 %. The intervals overlap throughout and nothing here is a difference. What the lifecycle table shows is *why*, and it is not that the loop is idle.

arm	proposed	regime / judge	T1 pass / inc. / fail	T2 pass / fail / open	canaries run / undecided	shipped
SenseLab, pre-seam4	9	5 / 4	7 / 2 / 0	2 / 2 / 0	3 / 3	0
SenseLab − canary, pre-seam4	8	4 / 4	4 / 3 / 1	1 / 2 / 0	0 / 0	1
SenseLab, post-seam5	16	12 / 4	5 / 10 / 1	6 / 3 / 7	4 / 3	0
SenseLab − canary, post-seam5	17	13 / 4	12 / 4 / 1	9 / 7 / 1	0 / 0	9
SenseLab, post-seam5b	24	19 / 5	14 / 9 / 1	15 / 8 / 1	6 / 4	0
SenseLab − canary, post-seam5b	16	13 / 3	10 / 5 / 1	11 / 4 / 0	0 / 0	11

Table 5: Fix lifecycle by run and arm. Tier 1: stand-in replay on the fix branch; Tier 2: the customer's replay of the case set; a canary grades live exposure in the fix's situation. *open*: Tier 2 replays still awaited when the cell ended. No canary reached a decision inside a 100-episode cell; no fix was reverted; 96–100% of proposals were drafted by the repair agent.

On the latest complete cut the loop proposed 24 fixes in the four SenseLab cells, 19 of them from regime shifts; 14 passed Tier 1, 15 received a Tier 2 verdict and passed, and **none shipped**, because no canary reached `min_outcomes` graded runs in its situation before the cell ended (the

canaries that started were still running at episode 100, with 1–4 canary-arm runs graded against a floor of 8). A running canary is not inert: half of the sessions in its situation read the fix branch while it runs, so an unshipped fix reaches behaviour in SenseLab cells too. The ablation without the canary shipped 11 of its 16 proposals on a passed Tier 2 — 8 on `ci-fix`, 3 on `support`. By family its moved and kept halves are 62 % and 87 % on `ci-fix` (SenseLab: 50 and 71) and 12 % and 56 % on `support` (SenseLab: 37 and 44); with 27–43 episodes per half per family none of these is a difference, and the confirmatory run is powered for one where this one is not.

What the shipped and proof-passing fixes *say* is not in doubt. Hand-coding the fix text against the true change, 20 of the 24 regime fixes that passed their proof set on this cut encoded an unconditional flip ("always Y") where the change was conditional ("Y when a, else X"); on `ci-fix`, 19 of 19. That is the central finding of the development seeds and the subject of finding 9 (§5): the proof set graded a replay on the *outcome* it reported, and this agent — like any agent with a retry budget — reports `success` for an episode that tried the stale action, was refused, recorded the attempt and recovered. Graded on that, a fix that flipped the rule wholesale passed every case in which the agent could recover, which on `ci-fix` is every case. The fix to the engine (a proof case passes only on a *clean* success, with no recorded failed attempt) is live and under test. Why a wholesale flip on the branch does not show as a kept-half loss at this sample is open: the agent follows a retrieved rule through a recommendation, not blindly, and the confirmatory analysis reads the judge's followed-memory criterion per episode to separate "the rule was wrong" from "the rule was wrong and followed".

4.3 SenseLab by engine cut

engine cut	moved half	kept half	unchanged	escalated	fixes proposed	Tier 2 passed	shipped
pre-seam4 (‘rsi-c1-rsi‘)	36 % (27/75)	65 % (42/65)	96 % (134/139)	6 % (22/399)	9	2	0
post-seam5 (‘rsi-c2‘)	47 % (35/75)	60 % (39/65)	99 % (138/140)	6 % (24/400)	16	6	0
post-seam5b (‘rsi-c3‘)	43 % (32/75)	60 % (39/65)	99 % (138/140)	8 % (30/400)	24	15	0

Table 6: SenseLab on successive cuts of the engine, same protocol and seeds. Fix counts are over the four SenseLab cells of each run. `pre-seam4`: amfs-internal #787/#788; `post-seam5`: #793, #794; `post-seam5b`: #795, #796; `post-seam8`: #797 (§5).

Table 6 is the same protocol on successive cuts of the engine and is the trace of §5 in numbers: the loop went from 9 proposals and 2 Tier 2 verdicts on the first cut to 24 and 15 on the third as its evidence path was repaired, while the outcome endpoints moved within noise. The cut under test now is the first on which the loop's proof set can see the conditional.

4.4 Cost

system	tokens/ep	agent $/ep	arm $/ep	memory s/ep	wall s/ep
Frontier alone	1,287	0.011	0.000	0.0	5
Context window	24,322	0.070	0.000	0.0	4
pgvector	4,475	0.027	0.000	0.0	9
pgvector + rerank	3,953	0.025	0.003	0.0	10
pgvector + outcomes (DIY)	5,406	0.029	0.005	0.0	9
SenseLab	6,262	0.031	0.000	12.2	22
SenseLab − canary	6,025	0.031	0.000	12.3	22
SenseLab − repair loop	6,537	0.032	0.000	10.5	20

Table 7: Cost and latency per episode. *agent $*: the agent's own calls; *arm $*: every other call the arm caused that is charged per episode (reflection, consolidation); the repair loop's own spend is on the account's evaluation ledger and not yet here. *memory s*: time in memory calls against the production API.

SenseLab's agent cost per episode (0.031) is within 15 % of the pgvector memories and 2.3× below the context window; its memory time (12 s per episode against the production API, under the harness's rate pacing) is the price of reading the evidence record rather than a vector index. The loop's own spend — repair agent, stand-in replays and judges — is charged to the account's evaluation ledger and is [not yet in this table; E6 in the preregistration].

4.5 Two probes on the earlier change set

When the team grows. On the unconditional change (set A, change at 20), with a fleet of six agents sharing one store: in the *pioneer* protocol one agent works alone for 60 episodes and five join; in the *newcomer* protocol five work and a sixth joins at 60. A joiner on SenseLab takes the post-change action on its first exposure to a changed class in 36/36 (pioneer) and 10/10 (newcomer) cases with zero stale picks; a joiner on pgvector + rerank does so in 31 % and 40 % with 17 % and 0 % stale picks and 16–20 % escalations. The inheritance is the outcome record, not the notes: the same notes are in both stores.

arm	veterans ep50-59 1st	veterans changed post 1st	joiner 1st-exp all	1st-exp changed	1st-exp unchanged	stale-pick (changed 1st-exp)	escalated
pgvector + rerank	0.75	0.28	0.68 [0.52,0.83] (n=90)	0.31 [0.15,0.50] (n=36)	0.93 [0.78,1.00] (n=54)	0.17	0.20
SenseLab	1.00	0.76	1.00 [1.00,1.00] (n=90)	1.00 [1.00,1.00] (n=36)	1.00 [1.00,1.00] (n=54)	0.00	0.00

Table 8: When the team grows (set A, change at 20, fleet of six sharing one store; *pioneer*: one agent works alone to episode 60, five join). First-attempt success of the joiners on their first exposure to each class after joining, with the veterans' rate before they joined; *stale-pick*: the joiner applied the pre-change action.

A student trained on the record (E7 at rung 0). Two Managed Models (`senselab-s1`) were trained on the dev-seed `support` traces of SenseLab and of the ablation without the repair loop (200 traces each), and evaluated on held-out seeds 101/113/127 (1,500 episodes). Alone, neither student takes the new action first (9 % and 0 % on the moved half); with SenseLab's outcome loop as its memory each does (33 % and 30 %), at 1/23 of the frontier model's cost per episode — and beats GPT-5.5 with the same memory (22 %) on the moved half at this rung, which is the contamination the evaluation ladder exists to expose: rung 0 uses the training classes. The SenseLab-trained student against the ablation-trained one is $-7/0/+18$ points across the three seeds — "small and reported as such" is the preregistered sentence. Nothing is claimed from rung 0; rungs 1–3, on classes and wordings the students never saw, need eval-only content that is registered but not yet written.

5 What the loop taught us about itself

The preregistration promised to report every prediction. Table 9 is that report, and the findings below are the record of what a benchmark that knows the true rule found in a shipped self-improvement loop — eight defects and two properties of the problem — none of which a retrieval benchmark could have seen. Seven of the defects were fixed in production between runs, with their deploy cuts recorded, and re-measured on the same protocol; the eighth was found while this version was being written. We think this section is the paper's main contribution; the loop's evidence path is where such systems fail, and none of these failures is specific to ours.

1. **The loop was never reached.** (Pilots 5b–5g.) Incidents open at three failing verdicts on one behaviour cluster; a fresh agent's judges had graded nothing, so no incident opened and no fix was proposed in 60 episodes. The regime-shift producer — an entry validated ≥ 3 times that now fails — is the producer a memory can drive without judges, and it is what every result here rests on.

2. **Sealing stalled under load.** A lock on the seal path held sessions open under the grid's concurrency; fixed (#780/#781) before any reported run.

3. **A first-exposure failure is not a regime shift.** (Amendment (k), pilot 5k; P1 falsified 3/3.) The shift producer fired on entries that had been *read*, not validated, so the first failure of a never-tested entry opened a canary that could grade nothing — the canary compares recurrence of a failure the control arm had never had. The guard now requires validation in the situation.

4. **On an unconditional change, demotion recovers almost everything.** (Amendment (l); P4.) A scripted maintainer writing the true rule five episodes after the change beat outcome learning alone by 2.5 (`support`) and 4.2 (`ci-fix`) points on 60 episodes of set A. That is the ceiling a repair loop can reach on such a change, and it is small. Change set C was written so that demotion has no signal: the entry is still right on the kept half.

5. **A regime fix had nothing to be proved against.** (Seam 4, #793; P6 met on the ablation.) Case sets were built from judge verdicts; a fix drafted from a shift had none, so Tier 2 could not be asked and `auto_after_replay` had nothing to ship on. The manifest's runs now seed the case set, and a judge-less case is graded on the replay's outcome.

6. **The agent spells one situation ten ways.** (Seam 5, #794; P7 recall 0.71, purity 0.92 against 0.80/0.90 predicted; seam 5b, #796, P7′ falsified: 0.40/0.67.) A canary scoped to a situation graded 2 of 15 in-situation runs because the agent labelled the same situation `csv export timeout`, `data export error`, `large export failing`. A server-side canonical resolves labels by stemmed tokens; the embedding second stage added for recall *lowered* it and merged `ci-fix` classes (`format check failure` $\sim$ `audit check failure` at 0.79). It is left on by decision, with its cost stated.

7. **A running canary swallowed every later fix's replays.** (Seam 7, #795; P8 met.) The router bucketed a Tier 2 replay session like any other; a replay for fix B sealed during A's canary was stamped onto A's branch, counted in A's arm, and reported `missing` to B. Replays are now re-stamped onto their own branch and excluded from every canary tally.

8. **The loop drafted fixes from its own tests.** (Seam 8, #797; P10 pending.) A Tier 2 replay that failed on a fix branch read to the regime pass as the entry failing on `main`: 13 of 40 fixes on the `post-seam5b` cut were drafted from a replay, in chains — a fix from the replay of a

fix from a replay — and one replay's spelling became a canonical that four served runs then joined. Replays are no longer traffic at the seal: no judge, no identity, no shift. (The first cut of this rule also excluded the labelled copy of a served inference, which is traffic; an automated reviewer caught it before merge.)

9. **The proof graded the wrong signal.** (Amendment (s), #797; P9(b) falsified 20/24, P11 pending.) See §4: an outcome-graded case passed a recovered success, so an unconditional flip passed a proof set built to catch it. A case now passes only on a clean success — no recorded failed attempt — and the change is dated by the case, not the deploy, so earlier case sets keep their semantics.

10. **Who grades the proof.** (Found while this version was being written; fix pending approval.) The clean-success rule of finding 9 applies to a case no judge graded. In this benchmark 174 of 175 proof cases carry a judge — the benchmark's own frozen rubric, which the continuous-judging tier had already run on the evidence trace — and that rubric says, by design, that "a run that failed one or more attempts before succeeding is a PASS". The proof tier therefore inherits the coarse signal through the judge as well as through the outcome, and finding 9's fix reaches one case in 175 here. The size of the leak is measurable: of the 555 case replays run since the `post-seam5b` cut, 479 passed, and 116 of those passes (24 %) carry a recorded failed attempt — a recovered success the proof counted as clean. The engine-side correction is to carry the clean-success requirement on every case built for a regime fix, whatever grades it, and enforce it after the verdict; it is put to the owner as a pull request, and P11 is re-targeted to the run after its cut. We record it here rather than after the fact because the run this version reports as pending will test P10 and not P11.

The canary inside a cell. Separately from the defects, no canary reached a decision inside any 100-episode cell (K1–K4 in the readouts): a situated canary grades only in-situation runs, and at `pct`=50 with a floor of 8 per arm it needs 16 in-situation graded sessions after it starts, which a cell with 4–10 post-change exposures of one class cannot supply. This is a property of the protocol, not a defect; the flagship long-horizon experiment (200 episodes, fleet 10) is sized for it, and Table 4's "− canary" row is the reading of what ships when the gate is the replay alone.

P	Where	Prediction (dev seeds, set C unless stated)	Verdict
P1	(k) 5k	a shift-drafted fix is graded on runs that read the entry	falsified 3/3
P2	(l) c1	− canary ships a regime fix in every cell; recurrence falls	not met (1/4)
P3	(l) c1	SenseLab's canary reaches a decision in-cell	not met (K1–K4)
P4	(l) 5A	maintainer oracle within 3 pts of outcome learning, set A	+2.5 / +4.2
P5	(l)	judges-only recovers ≥ 0.7 of oracle recovery	pending
P6	(o) c2	every Tier 1-passed regime fix gets a Tier 2 verdict	met (ablation); falsified by seam 7 (SenseLab)
P7	(o) c2	situation recall ≥ 0.80, purity ≥ 0.90	0.71 / 0.92
P7′	(r) c3	embedding stage: recall ≥ 0.80, purity ≥ 0.90	falsified 0.40 / 0.67
P8	(q) c3	replays during a canary are re-stamped and graded	met (26/26)
P9(a,c)	(q) c3	proof set of 3–6 cases; kept-case failures occur	met
P9(b)	(q) c3	$\leq$ 25 % of proof-passing regime fixes non-conditional	falsified 83 %
P10	(s) c4	no fix drafted from a replay; no alias from a replay	pending
P11	(s) c4	non-conditional share $<$ 50 %; a `ci-fix` kept-case failure	not reachable on c4 (finding 10); re-targeted

Table 9: The predictions ledger. Every preregistered prediction on the engine so far, with the amendment that made it, the run that tested it and its verdict. Pending rows are being tested on the run this version reports as open.

6 Pending, and what we expect

The run on the current cut (`post-seam8`: findings 8 and 9 live) is in progress at the time of writing, on the same protocol and seeds. The preregistered predictions are: (P10) no fix has a manifest run that is a replay, and no situation alias is created by a replay — falsified by one; (P11) the share of proof-passing regime fixes that do not encode the conditional falls below 50 % from 83 %, at least one regime fix fails Tier 2 on a kept case on `ci-fix`, and the number of ships falls — falsified by a share $\geq$ 70 %, or no `ci-fix` failure with four or more `ci-fix` regime fixes reaching Tier 2. Finding 10 means the pending run tests P10 only; P11 is re-targeted to the first run after the judge-graded proof carries the clean-success requirement, with its prediction unchanged. If P11 then holds, the "− canary" row of Table 4 should lose ships and gain kept-half success; if the loop then writes conditional rules, SenseLab's kept half should rise toward the retrieval baselines' without losing the moved half. If P11 is falsified, the repair agent, not the proof, is where the conditional is lost, and the paper will say so.

7 Related work

Self-improvement. Self-Refine [9] and STaR [10] improve a model's outputs or weights from its own feedback within a task or a training run. The Gödel machine lineage [11, 12] and AlphaEvolve [13] modify the improver itself, gated by a proof or a benchmark score, in a sandbox. The engine here is the deployed case: the object it modifies is what every live agent acts on, the gate is live traffic rather than a held-out score, and the discipline — manifest, tiers of proof, staged exposure, reversibility, an audit trail — is the contribution rather than the search. The "fixed budget, held-out, sustained, generalising" conditions that the preregistration adopts for any claim

of self-improvement (E5) follow those proposed for Level-1 AI research agents [cite: verify the Weco reference before v0.2].

Release engineering. Canary release and controlled online experiments [14] are the discipline the loop borrows for level 1; A/A tests and look-schedule inflation are the standard hazards, and the preregistration's A/A probe measures the empirical false-pass rate of the canary's test under the schedule the harness uses.

Benchmarks. τ-bench and τ^2-bench [15, 16] evaluate tool-using agents against a policy that stays fixed across tasks; our public track (E4) amends the policy between passes to ask this paper's question on a public task set. Continual-learning benchmarks for language models measure forgetting in weights; the object here is what a deployed agent acts on, and the failure mode is remembering.

Agent memory, as substrate and as baseline. Reflexion [2] and Voyager [3] write verbal reflections and skills back for later episodes; Generative Agents [4] and MemGPT [5] manage a store with retrieval and summarisation; Mem0 [6] and Zep [7] are production memory layers evaluated on conversational recall (LoCoMo, LongMemEval [8]). All are evaluated on retrieving what was stored; none tests, gates or reverts a change to the store, and none is measured on a change of the world after storage. They are the substrate a self-improving engine operates on and the baselines it is measured against here; our earlier report [1] measured level 0 against these designs under a regime change.

8 Threats to validity

- **Development seeds.** Every number is from two seeds on two families, chosen for development. The confirmatory seeds run once after the freeze; nothing here is a confirmed difference, and the intervals say so.
- **One model, one effort level.** GPT-5.5 at low effort. The preregistration includes Claude Opus 5; it has not run on set C.
- **The benchmark's outcome is coarse.** The harness reports `success` for an episode solved within three attempts; that is what the customer reports, and it hid finding 9 from the proof set for two runs. The clean-success rule reads the attempts the agent already records; a customer whose agent never records attempts sees no change and gets no protection.
- **The engine changed between runs.** Seven fixes were deployed during the study. Every result carries its cut; latency claims are within one cut only; and the changes are to the loop's evidence path, not to any threshold constant, which the preregistration forbids.
- **Hand-coding.** The "non-conditional" share (P9(b), P11) is a reading of the shipped text against the known change by one coder; the confirmatory run will use two, blind to arm.
- **The harness routes the canary.** The production router runs in the MCP gateway; the harness applies the same hash itself (parity-tested in CI) because it talks to the HTTP API.
- **The context window is a strong baseline where it fits.** On `ci-fix` the whole cell fits in 24k tokens and the window sees the change; it is the right comparison for short horizons and the wrong one for the 200-episode fleet, where it cannot fit.

- **We built the system.** The preregistration, the amendments with their falsifiers, the cut records and the ledger exist so that a reader can check what was predicted before each run rather than take our reading of it.

A Reproducibility

Runs. Harness arm names, cell ids and command lines: baselines `rsi-c1-tier1` (`none`, `pgvector+rerank`, `senselab`), `rsi-c1-baselines` (`pgvector`, `pgvector-diy+outcomes`), `rsi-c1-baselines-cw` (`raw-traces`); SenseLab `rsi-c1-rsi`, `rsi-c2`, `rsi-c3`, `rsi-c4` (`senselab-rsi`, `senselab-rsi-nocanary`). Cell id `<family>|<arm>|gpt-5.5|s<seed>|e100-d300-n0.0-f0-a1-c30-x0.5-oid-vC`. Command:

```
python -m continual_learning.runner --run <run> --scenarios support ci-fix --arms <arms> --models
gpt-5.5 --seeds 11 23 --fleet 1 --episodes 100 --change-at 30 --change-focus 0.5 --change-variant
C --opaque-ids --workers 2
```

with `CL_RSI_TIER2=1` and a Tier 2 replay receiver reachable from the service.

Engine cuts. `pre-seam4`: amfs-internal #787/#788. `post-seam5`: #793 (07:00:22Z), #794 (08:27:24Z, 2026-09-27). `post-seam5b`: #795 (12:50:24Z), #796 (13:39:13Z). `post-seam8`: #797 (09:14:17Z, 2026-09-28). Each cut's Cloud Run revision is in `HYPOTHESIS_RSI.md`.

Analysis. `python -m benchmarks.continual_learning.analysis.rsi --senselab-run rsi-c3 --paper` writes `results/rsi-report/REPORT.md` and `report.json`; `analysis.fill_rsi_paper` fills this document from it. The preregistration, amendments, readouts (`HEADROOM_RSI.md`) and change sets are published with the paper.

References

[1] SenseLab Research. From memory to recursive self-improvement: agents that learn from outcomes, agents that only remember, and a model trained on what they learned, when the tasks never repeat and the rules change. Preprint, 2026.

[2] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao. Reflexion: Language agents with verbal reinforcement learning. NeurIPS 2023.

[3] G. Wang et al. Voyager: An open-ended embodied agent with large language models. TMLR 2024 (arXiv:2305.16291).

[4] J. S. Park et al. Generative agents: Interactive simulacra of human behavior. UIST 2023.

[5] C. Packer et al. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560, 2023.

[6] P. Chhikara et al. Mem0: Building production-ready AI agents with scalable long-term memory. arXiv:2504.19413, 2025.

[7] P. Rasmussen et al. Zep: A temporal knowledge graph architecture for agent memory. arXiv:2501.13956, 2025.

[8] D. Wu et al. LongMemEval: Benchmarking chat assistants on long-term interactive memory. ICLR 2025.

[9] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. NeurIPS 2023.

[10] E. Zelikman, Y. Wu, J. Mu, N. D. Goodman. STaR: Bootstrapping reasoning with reasoning. NeurIPS 2022.

[11] J. Schmidhuber. Gödel machines: Fully self-referential optimal universal self-improvers. In *Artificial General Intelligence*, Springer, 2007.

[12] J. Zhang, S. Hu, C. Lu, R. Lange, J. Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. arXiv:2505.22954, 2025.

[13] A. Novikov et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131, 2025.

[14] R. Kohavi, D. Tang, Y. Xu. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press, 2020.

[15] S. Yao, N. Shinn, P. Razavi, K. Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv:2406.12045, 2024.

[16] V. Barres et al. τ^2-bench: Evaluating conversational agents in a dual-control environment. arXiv:2506.07982, 2025.